

RAPPORT TECHNIQUE

GreenFinance-Scorer

Étape 3 — Base de données (schéma pivot complet)

Trace technique complète des sous-tâches 3.1 à 3.9

Projet	GreenFinance-Scorer — plateforme de scoring ESG et carbone pour entreprises
Dépôt	<code>github.com/cheikhmama/GreenFinance-Scorer</code>
Étape documentée	Étape 3 / 21 — Base de données (schéma pivot complet), version révisée après retour utilisateur
Dépend de	Étape 2 (validée, cf. rapport dédié)
Sous-tâches	9 (Prompts 3.1 → 3.9), toutes validées
Date du rapport	12 août 2026
État git	60 fichiers modifiés/nouveaux, non committés au moment du rapport (dont 56 hérités de l'Étape 2, elle-même non committée)
Statut final	TOUS CRITÈRES VALIDÉS

1. Résumé exécutif

Ce document trace intégralement l'exécution de l'Étape 3 du plan d'implémentation GreenFinance-Scorer : la traduction fidèle du diagramme de classes (Passage B) en schéma SQLAlchemy/PostgreSQL, enrichie de trois clarifications posées avant l'implémentation — l'anonymat de l'auditeur vis-à-vis de l'Entreprise (le champ `auditeur_id` reste enregistré pour l'accountability mais ne doit jamais être exposé à un compte Entreprise), l'ouverture des configurations de pondération personnalisées à l'Investisseur, au Chercheur et à l'Institution (pas réservées à l'Investisseur), et un modèle complet d'investissement (devise, taux de change figé à la transaction, durée FIXE/OUVERTE).

Les 9 sous-tâches (Prompts 3.1 à 3.9) ont été menées à terme avec succès. Un incident technique a nécessité diagnostic et correction durant le Prompt 3.9 : la migration générée par `alembic revision --autogenerate` référence `sqlmodel.sql.sqltypes.AutoString` sans importer le module `sqlmodel`, ce qui aurait levé une `NameError` à l'exécution — corrigé dans le fichier de migration et dans le template `script.py.mako` pour que les migrations futures en soient exemptes. L'édition de `alembic/env.py` et le fichier de migration autogénéré ont par ailleurs introduit 8 écarts de convention d'imports, corrigés par `ruff check --fix` et revérifiés (cycle de migration, mypy, suite de tests complète).

Entités créées	15 tables (14 entités du diagramme de classes + 1 table de rattachement <code>chercheur_institution</code>) ; 9 enums (7 hérités du diagramme + 2 nouveaux : <code>DevisePosition</code> , <code>TypeDureeInvestissement</code>)
Résultat des tests	71/71 tests passés (54 dédiés au schéma pivot : 9 unitaires sur les enums + 45 d'intégration sur les modèles/reliations, adossés à la base PostgreSQL réelle), couverture 98 % sur <code>app/</code>
Qualité du code	<code>ruff check .</code> et <code>mypy app/</code> : aucune erreur (55 fichiers source)
Migration Alembic	<code>b5f953dddf56</code> — cycle <code>upgrade head</code> → <code>downgrade base</code> → <code>upgrade head</code> validé deux fois, y compris sur base neuve (<code>docker compose down -v</code>)
Infrastructure	Stack complète (db + redis + api) revalidée après reconstruction de la base ; <code>/health</code> → <code>{"status":"ok","database":"connected"}</code>

Table des matières

- 1. Résumé exécutif
- 2. Prompt 3.1 — Enums communs
- 3. Prompt 3.2 — Utilisateur et spécialisations
- 4. Prompt 3.3 — Entreprise
- 5. Prompt 3.4 — RapportESG
- 6. Prompt 3.5 — IndicateurESG, Donneecarbone, PreuveDocumentaire
- 7. Prompt 3.6 — ConfigurationPonderation, ScoreESG
- 8. Prompt 3.7 — Portefeuille et investissements
- 9. Prompt 3.8 — AvisAudit, SignalementEcart, Notification
- 10. Prompt 3.9 — Migration Alembic complète et test d'intégrité
- 11. Bilan final — critères de passage à l'Étape 4
- 12. Annexes — fichiers créés, migration, arborescence

2. Prompt 3.1 — Enums communs

OBJECTIF

Que chaque énumération du diagramme de classes existe en Python, une seule fois, y compris les deux nouvelles requises par la clarification sur le modèle d'investissement.

CONFIGURATION MISE EN PLACE

`app/core/enums.py` — 9 enums, valeurs en français :

- ✓ `StatutRapport` — 7 valeurs (ENVOYE → DEMANDE_CORRECTION)
- ✓ `TypeRapport`, `CanalDepot`, `Pilier`, `MethodeDonnee`, `DecisionAudit`, `Role` — repris du schéma pivot initial, non dupliqués
- ✓ `DevisePosition` (MRU, USD, EUR) et `TypeDureeInvestissement` (OUVERTE, FIXE) — les deux nouvelles énumérations de cette révision
- ✓ `sa_enum_column()` — colonne stockée en `VARCHAR + CHECK ( native_enum=False )` plutôt qu'en type ENUM natif PostgreSQL, pour éviter un conflit de création quand une même énumération (ex. `MethodeDonnee`) est réutilisée sur plusieurs tables

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ pytest tests/unit/test_enums.py -v
tests/unit/test_enums.py::test_statut_rapport_values PASSED
tests/unit/test_enums.py::test_type_rapport_values PASSED
tests/unit/test_enums.py::test_canal_depot_values PASSED
tests/unit/test_enums.py::test_pilier_values PASSED
tests/unit/test_enums.py::test_methode_donnee_values PASSED
tests/unit/test_enums.py::test_decision_audit_values PASSED
tests/unit/test_enums.py::test_role_values PASSED
tests/unit/test_enums.py::test_devise_position_values PASSED
tests/unit/test_enums.py::test_type_duree_investissement_values PASSED
9 passed in 0.25s
```

CRITÈRES DE VALIDATION

- ✓ 9/9 enums importables, valeurs exactes vérifiées une à une (y compris les deux ajouts)
- ✓ Aucun doublon avec un enum déjà créé à l'Étape 2
- ✓ Correspondance exacte avec le diagramme de classes révisé

3. Prompt 3.2 — Utilisateur et spécialisations

OBJECTIF

Un seul modèle de compte, discriminé par rôle — jamais une table séparée par sous-rôle.

CONFIGURATION MISE EN PLACE

app/auth/models.py :

- ✓ Utilisateur (table utilisateur) — id, email (unique, indexé), mot_de_passe_hache, role, date_creation, actif (défaut True)
- ✓ InstitutionProfil (table institution_profil) — relation 1-1 avec Utilisateur (utilisateur_id unique), champ quota_export
- ✓ ChercheurInstitution (table chercheur_institution) — table de rattachement plusieurs-à-plusieurs Chercheur ↔ Institution

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ pytest tests/integration/test_models_auth.py -v
tests/integration/test_models_auth.py::test_creation_utilisateur_par_role[ADMINISTRATEUR] PASSED
tests/integration/test_models_auth.py::test_creation_utilisateur_par_role[ENTREPRISE] PASSED
tests/integration/test_models_auth.py::test_creation_utilisateur_par_role[AUDITEUR] PASSED
tests/integration/test_models_auth.py::test_creation_utilisateur_par_role[INVESTISSEUR] PASSED
tests/integration/test_models_auth.py::test_creation_utilisateur_par_role[CHERCHEUR] PASSED
tests/integration/test_models_auth.py::test_creation_utilisateur_par_role[INSTITUTION] PASSED
tests/integration/test_models_auth.py::test_rattachement_chercheur_institution PASSED
tests/integration/test_models_auth.py::test_email_unique_constraint PASSED
8 passed in 0.37s
```

CRITÈRES DE VALIDATION

- ✓ Création d'un Utilisateur réussie pour chacun des 6 rôles
- ✓ Rattachement Chercheur/Institution fonctionnel
- ✓ Contrainte d'unicité sur email vérifiée (violation → IntegrityError)
- ✓ Aucune table séparée par sous-rôle

4. Prompt 3.3 — Entreprise

OBJECTIF

Une Entreprise avec compte utilisateur optionnel, et un montant minimum d'investissement — champ requis par la clarification sur le modèle d'investissement, absent de la version initiale du schéma pivot.

CONFIGURATION MISE EN PLACE

app/company/models.py — Entreprise (table entreprise) : id, nom, secteur, pays, logo (optionnel), description (optionnel), site_officiel (optionnel), actif (défaut True), utilisateur_id (clé étrangère optionnelle), montant_minimum_investissement (optionnel — nul signifie « aucun minimum imposé »).

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ pytest tests/integration/test_models_company.py -v
tests/integration/test_models_company.py::test_creation_sans_montant_minimum_et_sans_compte PASSED
tests/integration/test_models_company.py::test_creation_avec_montant_minimum PASSED
tests/integration/test_models_company.py::test_montant_minimum_nul_distinct_de_zero PASSED
tests/integration/test_models_company.py::test_utilisateur_id_inexistant_rejete PASSED
4 passed in 0.37s
```

CRITÈRES DE VALIDATION

- ✓ Création avec et sans montant_minimum_investissement ; une Entreprise sans compte utilisateur reste valide (dépôt automatique par le pipeline, sans compte préexistant)
- ✓ Un utilisateur_id inexistant est rejeté (IntegrityError)
- ✓ montant_minimum_investissement nul est un cas valide et distinct de zéro, vérifié explicitement par un test dédié

5. Prompt 3.4 — RapportESG

OBJECTIF

Le cycle de vie documentaire, sept statuts. `auditeur_id` reste présent pour l'accountability, mais ne sera jamais exposé à l'Entreprise (règle d'accès à implémenter aux Étapes 9-11, pas dans ce prompt).

CONFIGURATION MISE EN PLACE

`app/ingestion/models.py` — RapportESG (table `rapport_esg`): `id`, `entreprise_id` (clé étrangère obligatoire), `type`, `canal`, `date_depot`, `statut` (défaut `ENVOYE`), `fichier_source`, `auditeur_id` (clé étrangère vers `utilisateur`, optionnelle, champ interne).

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ pytest tests/integration/test_models_ingestion.py::test_creation_rapport_lie_a_entreprise_statut_par_default \
\
    tests/integration/test_models_ingestion.py::test_entreprise_id_obligatoire \
    tests/integration/test_models_ingestion.py::test_entreprise_id_inexistant_rejete \
    tests/integration/test_models_ingestion.py::test_sept_statuts_tous_atteignables -v
test_creation_rapport_lie_a_entreprise_statut_par_default PASSED
test_entreprise_id_obligatoire PASSED
test_entreprise_id_inexistant_rejete PASSED
test_sept_statuts_tous_atteignables[ENVOYE] PASSED
test_sept_statuts_tous_atteignables[EN_EXTRACTION] PASSED
test_sept_statuts_tous_atteignables[AFFECTE_AUDITEUR] PASSED
test_sept_statuts_tous_atteignables[EN_VALIDATION] PASSED
test_sept_statuts_tous_atteignables[VALIDE] PASSED
test_sept_statuts_tous_atteignables[REJETE] PASSED
test_sept_statuts_tous_atteignables[DEMANDE_CORRECTION] PASSED
10 passed in 0.34s
```

CRITÈRES DE VALIDATION

- ✓ Création liée à une Entreprise existante, statut par défaut `ENVOYE`
- ✓ `entreprise_id` obligatoire (`IntegrityError` si nul ou inexistant), `auditeur_id` optionnel
- ✓ Les sept statuts sont tous atteignables (test paramétré sur l'énumération complète)

6. Prompt 3.5 — IndicateurESG, DonneeCarbone, PreuveDocumentaire

OBJECTIF

Les données produites exclusivement par le pipeline automatique (Étapes 4-7) : l'Auditeur les consulte et les valide via `AvisAudit` (Prompt 3.8), il ne les crée jamais lui-même.

CONFIGURATION MISE EN PLACE

`app/ingestion/models.py`, ajouts :

- ✓ `PreuveDocumentaire` — `nom_document`, `annee`, `nombre_pages_total`, `page_debut`, `page_fin`, `pdf_extrait_genere`
- ✓ `IndicateurESG` — `rapport_id`, `pilier`, `code`, `valeur`, `unite`, `methode`, `preuve_id`
- ✓ `DonneeCarbone` — `scope`, `categorie_ges` (optionnel), `valeur_tonnes_co2e`, `annee`, `methode`, `score_qualite_pcaf` (borné 1-5, validé au niveau Pydantic via `Field(ge=1, le=5)`), `preuve_id`

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ pytest tests/integration/test_models_ingestion.py -v -k "indicateur or pcaf or suppression"
test_indicateur_et_donnee_carbone_lies_au_meme_rapport PASSED
test_score_qualite_pcaf_hors_bornes_rejete[0] PASSED
test_score_qualite_pcaf_hors_bornes_rejete[6] PASSED
test_score_qualite_pcaf_hors_bornes_rejete[-1] PASSED
test_suppression_rapport_reference_echoue_proprement PASSED
5 passed, 10 deselected in 0.37s
```

POINT D'ATTENTION TESTÉ EXPLICITEMENT

`SQLModel ( table=True )` ne déclenche pas la validation Pydantic (`ge / le`) sur une construction directe

`Model(**kwargs)` — il faut passer par `.model_validate()` pour l'exercer réellement.

`test_score_qualite_pcaf_hors_bornes_rejete` documente ce point et l'exploite pour vérifier le rejet effectif des valeurs 0, 6 et -1.

CRITÈRES DE VALIDATION

- ✓ Création liée à un même `RapportESG` ; `score_qualite_pcaf` hors bornes rejeté
- ✓ Suppression d'un `RapportESG` encore référencé : `IntegrityError`, jamais de cascade silencieuse des indicateurs/données carbone associés (comportement documenté explicitement dans le test)
- ✓ Ces trois tables restent le seul point d'entrée des données ESG/carbone — aucun champ ni table ne permet à un Auditeur d'y écrire directement une valeur extraite manuellement

7. Prompt 3.6 — ConfigurationPonderation, ScoreESG

OBJECTIF

Configuration de référence vs. personnalisée, désormais ouverte à Investisseur, Chercheur et Institution — clarification centrale de cette révision.

CONFIGURATION MISE EN PLACE

app/scoring/models.py , ajouts :

- ✓ ConfigurationPonderation — nom , version , fichier_yaml , date_creation , utilisateur_id (optionnel — nul pour la configuration de référence maintenue par l'Administrateur, renseigné pour toute personnalisation quel que soit le rôle du créateur)
- ✓ ScoreESG — rapport_id , configuration_id , valeur_globale , score_environnement , score_social , score_gouvernance , date_calcul

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ pytest tests/integration/test_models_scoring.py -v
test_configuration_personnalisee_par_role[INVESTISSEUR] PASSED
test_configuration_personnalisee_par_role[CHERCHEUR] PASSED
test_configuration_personnalisee_par_role[INSTITUTION] PASSED
test_rapport_accepte_plusieurs_scores_un_par_configuration PASSED
test_utilisateur_id_nul_reserve_a_la_reference PASSED
5 passed in 0.53s
```

CRITÈRES DE VALIDATION

- ✓ Configuration personnalisée créée avec succès par un Investisseur, un Chercheur et une Institution — les trois réussissent sans distinction de rôle au niveau du modèle (test paramétré)
- ✓ Un RapportESG accepte plusieurs ScoreESG (un par configuration appliquée)
- ✓ utilisateur_id nul reste réservé à la configuration de référence — « une seule référence à la fois » documenté comme règle applicative (couche service, Étape 12), pas une contrainte de base à ce stade

8. Prompt 3.7 — Portefeuille et investissements

OBJECTIF

Le schéma complet pour la création de portefeuille et l'ajout de positions — devise, conversion figée, durée. Le formulaire, la conversion en temps réel affichée à l'écran et le calcul du taux appartiennent à l'Étape 16 ; ce prompt ne pose que le schéma qui les rendra possibles.

CONFIGURATION MISE EN PLACE

`app/investor/models.py` — `Portefeuille` et `PositionPortefeuille` (`montant_investi`, `devise`, `taux_change_utilise` figé, `montant_converti`, `type_duree`, `date_debut` / `date_fin`). Validations Pydantic (`model_validator`, `validate_assignment=True`):

- ✓ `montant_investi` ≥ `entreprise.montant_minimum_investissement` quand renseigné (via le contexte de validation)
- ✓ **FIXE** : `date_debut` ≥ maintenant ; `date_fin` ≤ `date_debut` + 1 an ; `date_fin` > `date_debut`
- ✓ **OUVERTE** : `date_fin` optionnelle à la création, si renseignée (fermeture) doit être postérieure à `date_debut` — appliqué aussi bien à la création qu'à une fermeture ultérieure grâce à `validate_assignment=True`

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ pytest tests/integration/test_models_investor.py -v
test_position_sous_le_minimum_requis_rejetee PASSED
test_position_fixe_date_fin_plus_dun_an_rejetee PASSED
test_position_fixe_date_debut_dans_le_passe_rejetee PASSED
test_position_fixe_valide_est_acceptee PASSED
test_position_ouverte_sans_date_fin_puis_fermeture PASSED
test_position_ouverte_fermeture_avec_date_fin_anterieure_rejetee PASSED
test_taux_change_fige_independant_entre_positions PASSED
test_suppression_entreprise_referencee_par_position_echoue PASSED
8 passed in 0.43s
```

CRITÈRES DE VALIDATION

- ✓ Position sous le minimum requis rejetée ; **FIXE** > 1 an rejetée ; **FIXE** avec `date_debut` passée rejetée ; **OUVERTE** réussit avec `date_fin` nulle puis sa fermeture ultérieure réussit si postérieure à `date_debut`
- ✓ Deux positions créées le même jour, même devise, taux différents : chacune conserve son propre taux — aucun recalcul rétroactif (`test_taux_change_fige_independant_entre_positions`)
- ✓ Suppression d'une Entreprise référencée par une position échoue proprement (`IntegrityError`), jamais de cascade silencieuse

9. Prompt 3.8 — AvisAudit, SignalementEcart, Notification

OBJECTIF

Le mécanisme d'audit et de correction ciblée, avec la règle d'anonymat de l'auditeur explicitée.

CONFIGURATION MISE EN PLACE

- ✓ `AvisAudit` (`app/audit/models.py`) — `rapport_id` , `auditeur_id` , `decision` , `commentaire` (optionnel), `date_avis`
- ✓ `SignalementEcart` (`app/ingestion/models.py`) — `indicateur_id` (clé étrangère vers `indicateur_esg` , jamais vers `rapport_esg`), `entreprise_id` , `nature_ecart` , `statut` (défaut `OUVERT`), `date_signalement`
- ✓ `Notification` (`app/core/models.py`) — `utilisateur_id` , `message` , `type` , `date_envoi` , `lu` (défaut `False`)

COMMENTAIRE DE DOCUMENTATION VÉRIFIÉ PAR RELECTURE

Au-dessus de `AvisAudit.auditeur_id` , présent mot pour mot dans le code source : « Champ interne réservé à l'accountability et au suivi de performance des auditeurs (Espace Administrateur). Ne doit JAMAIS être exposé par une route ou un schéma de réponse accessible à un compte Entreprise — règle d'accès à faire respecter strictement à partir de l'Étape 9. » — note à l'intention des développeurs futurs, humains ou Claude Code, pas vérifiée par un test automatisé conformément à la spécification.

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ pytest tests/integration/test_models_audit.py -v
test_creation_avis_audit PASSED
test_creation_signalement_ecart_ciblant_un_indicateur PASSED
test_indicateur_id_doit_referencer_un_indicateur_pas_un_rapport PASSED
test_creation_notification PASSED
4 passed in 0.35s
```

CRITÈRES DE VALIDATION

- ✓ Création d'un `AvisAudit` , d'un `SignalementEcart` ciblant un indicateur précis, d'une `Notification`
- ✓ `SignalementEcart.indicateur_id` cible bien un indicateur — passer l'id d'un rapport à la place est rejeté par la contrainte de clé étrangère (`IntegrityError`), jamais uniquement un rapport entier

10. Prompt 3.9 — Migration Alembic complète et test d'intégrité

OBJECTIF

Le schéma complet, révisé, s'applique sur une base fraîche — la migration Alembic devient la seule source de vérité, remplaçant le `create_all` de secours utilisé pendant les Prompts 3.2 à 3.8.

CONFIGURATION MISE EN PLACE

- ✓ `alembic/env.py` — `target_metadata` câblé sur `SQLModel.metadata`, tous les modules de modèles importés pour effet de bord (enregistrement des tables)
- ✓ Base réinitialisée (`docker compose down -v` && `docker compose up -d db`) pour qu' `autogenerate` parte d'un état vide et capture le schéma complet, pas un diff vide contre les tables déjà créées via `create_all` lors des prompts précédents
- ✓ `alembic revision --autogenerate` → `b5f953dddf56_schema_pivot_complet_etape_3.py`, 15 tables détectées (14 entités + `chercheur_institution`)
- ✓ `tests/integration/conftest.py` basculé : la fixture de session applique désormais `alembic upgrade head` (idempotent) au lieu de `SQLModel.metadata.create_all()`

ERREUR — IMPORT SQLMODEL MANQUANT DANS LA MIGRATION AUTOGÉNÉRÉE

Le fichier généré par `alembic revision --autogenerate` référence `sqlmodel.sql.sqltypes.AutoString()` pour chaque colonne texte, mais n'importe jamais le module `sqlmodel` — le template par défaut d'Alembic (`script.py.mako`) n'ajoute que `sqlalchemy as sa`.

```
NameError: name 'sqlmodel' is not defined
```

CORRECTION

`import sqlmodel` ajouté au fichier de migration généré, et au template `script.py.mako` lui-même (à côté de `import sqlalchemy as sa`) pour que les prochaines migrations autogénérées (Étapes 4+) n'aient pas à subir la même correction manuelle.

ERREUR SECONDAIRE — IMPORTS NON TRIÉS (RUFF)

`alembic/env.py` (nouveaux imports de modules de modèles) et le fichier de migration autogénéré (template Alembic par défaut : `from typing import Sequence, Union`) ne respectent pas les règles `I001` / `UP035` / `UP007` du projet — 8 erreurs détectées par `ruff check .`

CORRECTION

`ruff check . --fix` — 9 corrections appliquées (tri des imports, `Sequence` importé depuis `collections.abc`, syntaxe `X | Y`). Cycle `downgrade base` → `upgrade head` et suite de tests rejoués intégralement après correction pour confirmer qu'aucune modification fonctionnelle n'a été introduite par l'auto-fix.

COMMANDES EXÉCUTÉES ET RÉSULTATS

```
$ docker compose down -v && docker compose up -d db          # base neuve
Volume green-finance-scorer_pgdata Removed / Created
Container green-finance-scorer-db-1 Started

$ alembic upgrade head
Running upgrade -> 440af0be0e05, initial empty migration
Running upgrade 440af0be0e05 -> b5f953dddf56, schema pivot complet (Etape 3)

$ alembic downgrade base && alembic upgrade head
Running downgrade b5f953dddf56 -> 440af0be0e05, schema pivot complet (Etape 3)
Running downgrade 440af0be0e05 -> , initial empty migration
```

```
Running upgrade -> 440af0be0e05, initial empty migration
Running upgrade 440af0be0e05 -> b5f953dddf56, schema pivot complet (Etape 3)
# cycle rejoué une seconde fois après le fix ruff – même résultat

$ pytest tests/integration/test_schema_integrity.py -v
test_scenario_complet_schema_pivot_relations_bidirectionnelles PASSED
1 passed in 0.52s

$ pytest -q --cov=app
71 passed in 10.88s                                # couverture app/ : 98%

$ ruff check . && mypy app/
All checks passed!
Success: no issues found in 55 source files

$ curl http://localhost:8000/health
{"status":"ok","database":"connected"}
```

TEST D'INTÉGRITÉ — TESTS/INTEGRATION/TEST_SCHEMA_INTEGRITY.PY

Construit un scénario unique de bout en bout : Entreprise avec `montant_minimum_investissement` renseigné, RapportESG, IndicateurESG et DonneeCarbone avec leur PreuveDocumentaire, ScoreESG avec une ConfigurationPonderation de référence **et** une personnalisée créée par un Chercheur, Portefeuille avec une PositionPortefeuille en devise USD convertie (taux figé), une position FIXE avec dates valides, une position OUVERTE sans `date_fin`, un AvisAudit — et vérifie explicitement que chaque relation se résout dans les deux sens (ex. `rapport.indicateurs ↔ indicateur.rapport`, `config.scores ↔ score.configuration`, `entreprise.positions ↔ position.entreprise`).

CRITÈRES DE VALIDATION

- ✓ Migration générée couvrant les 15 tables, appliquée avec succès
- ✓ `upgrade head → downgrade base → upgrade head` sans erreur, vérifié deux fois (avant et après le fix ruff)
- ✓ `tests/integration/test_schema_integrity.py` passe, relations bidirectionnelles toutes vérifiées
- ✓ Vérification sur base fraîche : `docker compose down -v && docker compose up -d db && alembic upgrade head` réussie, puis suite complète rejouée avec succès
- ✓ Scénario complet, incluant les trois clarifications de la révision, représentable sans contournement

11. Bilan final — critères de passage à l'Étape 4

Critère (fin d'Étape 3, révisée)	Statut
Les 14 entités originales, plus les deux enums ajoutés, sont en place	VALIDÉ
Utilisateur reste une table unique à discriminant <code>role</code>	VALIDÉ
<code>ConfigurationPonderation.utilisateur_id</code> accepte Investisseur, Chercheur, Institution	VALIDÉ
<code>Entreprise.montant_minimum_investissement</code> existe et est correctement optionnel	VALIDÉ
<code>PositionPortefeuille</code> distingue montant saisi, devise, taux figé, montant converti, et applique les règles de durée (FIXE vs OUVERTE)	VALIDÉ
<code>AvisAudit.auditeur_id</code> est documenté comme champ interne, jamais exposé à l'Entreprise	VALIDÉ
Le pipeline automatique reste l'unique producteur d'IndicateurESG / DonneeCarbone / PreuveDocumentaire	VALIDÉ
La migration complète s'applique et se révoque sans erreur sur une base fraîche	VALIDÉ
Le test d'intégrité de bout en bout, incluant les trois clarifications, passe	VALIDÉ
<code>pytest</code> passe intégralement	VALIDÉ (71/71, couverture 98 %)

VALIDATION GLOBALE EXÉCUTÉE

```
$ docker compose down -v && docker compose up -d      # stack complète : db + redis + api
$ curl http://localhost:8000/health                  → {"status":"ok","database":"connected"} HTTP 200
$ alembic upgrade head                                → OK (base neuve, 15 tables créées)
$ pytest tests/ --cov=app                             → 71 passed, couverture 98%
$ ruff check . / mypy app/                           → propres (55 fichiers source)
```

Tous les critères de passage à l'**Étape 4 — Extraction documentaire (pipeline automatique)** sont remplis. Aucun commit git n'a été créé à ce stade — 60 fichiers modifiés/nouveaux restent en attente de validation par l'utilisateur avant commit (56 hérités de l'Étape 2, 4 propres au Prompt 3.9 de cette Étape 3 : `alembic/env.py`, `alembic/script.py.mako`, la migration `b5f953dddf56`, `tests/integration/test_schema_integrity.py` — le reste des modèles/tests de l'Étape 3, Prompts 3.1 à 3.8, provient d'une session antérieure).

12. Annexes

12.1 — LES 15 TABLES DU SCHÉMA PIVOT (MIGRATION B5F953DDDF56)

Table	Module	Prompt
utilisateur , institution_profil , chercheur_institution	app/auth/	3.2
entreprise	app/company/	3.3
rapport_esg	app/ingestion/	3.4
preuve_documentaire , indicateur_esg , donnee_carbone	app/ingestion/	3.5
configuration_ponderation , score_esg	app/scoring/	3.6
portefeuille , position_portefeuille	app/investor/	3.7
avis_audit	app/audit/	3.8
signalement_ecart	app/ingestion/	3.8
notification	app/core/	3.8

12.2 — FICHIERS CRÉÉS OU MODIFIÉS DURANT LE PROMPT 3.9

Catégorie	Détail
Migration	alembic/versions/b5f953dddf56_schema_pivot_complet_etape_3.py (nouveau)
Configuration Alembic	alembic/env.py (target_metadata), alembic/script.py.mako (fix import sqlmodel)
Tests	tests/integration/test_schema_integrity.py (nouveau), tests/integration/conftest.py (basculé sur alembic upgrade head)

12.3 — SUITE DE TESTS DU SCHÉMA PIVOT (71 TESTS, PROJET COMPLET)

tests/unit/test_enums.py	9 passed	(Prompt 3.1)
tests/integration/test_models_auth.py	8 passed	(Prompt 3.2)
tests/integration/test_models_company.py	4 passed	(Prompt 3.3)
tests/integration/test_models_ingestion.py	15 passed	(Prompts 3.4, 3.5, 3.8)
tests/integration/test_models_scoring.py	5 passed	(Prompt 3.6)
tests/integration/test_models_investor.py	8 passed	(Prompt 3.7)
tests/integration/test_models_audit.py	4 passed	(Prompt 3.8)
tests/integration/test_schema_integrity.py	1 passed	(Prompt 3.9)

	54 passed	— schéma pivot (Étape 3)
	17 passed	— hérités de l'Étape 2 (infra)
	71 passed	— suite complète du projet

12.4 — DÉCISIONS DE CONCEPTION (RAPPEL, NON RÉOUVERTES DURANT L'ÉTAPE 3)

- Utilisateur reste une table unique avec discriminant `role` (décision initiale, inchangée).
- L'extraction des données reste exclusivement produite par le pipeline automatique (Étapes 4 à 7) — l'Auditeur supervise et valide, il n'extrait jamais manuellement ; son identité n'est jamais exposée aux vues/API destinées à l'Entreprise, bien qu'enregistrée en base pour l'accountability.
- `ConfigurationPonderation` référence un `utilisateur_id` générique (Investisseur, Chercheur ou Institution), pas un `investisseur_id` spécifique.
- Le taux de change appliqué à un investissement est figé au moment de la transaction, jamais recalculé rétroactivement.